Narmada Billing System - Complete Easy Documentation

This document explains the full Narmada Billing System project in very simple words. The purpose of this document is: 1. To help understand the project later easily 2. To help fix errors quickly 3. To help deploy the project again 4. To help create new features safely 5. To help separate Production and Sandbox correctly Everything is explained step-by-step in easy language.

1. What This Project Is

```
This project is a billing and invoice management system.

Main purpose:
- Create invoices
- Manage companies
- Manage clients
- Generate bill PDFs
- Store invoice data
- Show dashboard statistics

The project is hosted completely on Cloudflare.

Main technologies used:
- HTML
- JavaScript
- Cloudflare Workers
- Cloudflare D1 Database
- Wrangler CLI

The project has:
- Frontend (UI that users see)
- Backend (API and database logic)
- Database
```

---

2. Understanding Frontend and Backend

```
Frontend:
Frontend means the design/UI users see in browser.

Frontend files:
- index.html
- config.js
- public folder
- favicon.ico

Frontend worker name:
narmada-billing-production

Frontend job:
- Show dashboard
- Show invoices
- Show companies
- Call backend APIs
- Show data to users

--------------------------------

Backend:
Backend means the server/API logic.

Backend worker name:
narmada-billing-api-production

Backend job:
- Receive requests
```

```
- Read database
- Save database
- Return data
- Handle APIs

Backend file:
backend/src/index.js
```

---

3. Production vs Sandbox

```
Production means:
REAL environment.

Production contains:
- Real company data
- Real invoices
- Real business data

Production workers:
- narmada-billing-production
- narmada-billing-api-production

Production database:
- narmada-billing-db

--------------------------------

Sandbox means:
TEST environment.

Sandbox contains:
- Testing data
- Fake invoices
- Safe environment

Sandbox workers:
- narmada-billing-sandbox
- narmada-billing-api-sandbox

Sandbox database:
- narmada-billing-sandbox

IMPORTANT:
Always test new features in sandbox first.
Never directly test on production.
```

---

4. Full Project Structure

```
Correct project structure:

narmada-billing/
■
■■■ backend/
■   ■■■ src/
■   ■   ■■■ index.js
■   ■■■ schema.sql
■
■■■ frontend/
■   ■■■ index.html
■   ■■■ config.js
■   ■■■ wrangler.toml
■   ■■■ favicon.ico
■   ■■■ public/
■
■■■ wrangler.toml

Explanation:
Root wrangler.toml:
```

```
Used for backend deployment.

Frontend wrangler.toml:
Used only for frontend deployment.

backend/src/index.js:
Main backend API file.

frontend/config.js:
Contains backend API URL.

schema.sql:
Contains database table structure.
```

---

5. Frontend config.js Explanation

```
This file is VERY IMPORTANT.

Location:
frontend/config.js

Production version:

const API = 'https://narmada-billing-api-production.jaypatni1710.workers.dev';

Sandbox version:

const API = 'https://narmada-billing-api-sandbox.jaypatni1710.workers.dev';

What this file does:
It tells frontend where backend API exists.

If wrong URL is written:
- Dashboard breaks
- Bills not load
- Clients not load
- Fetch errors occur

Common mistake:
Using localhost URL.

WRONG:
http://localhost:8787

This only works locally, not on production.
```

---

6. Backend wrangler.toml

```
Root wrangler.toml is used for backend.

Correct structure:

name = "narmada-billing-api"
main = "backend/src/index.js"

This tells Cloudflare:
- Worker name
- Main backend file

Production database:
narmada-billing-db

Sandbox database:
narmada-billing-sandbox

This file controls:
- Deployment
- Database binding
```

- Environment separation

---

7. Frontend wrangler.toml

Frontend MUST have separate wrangler.toml.

Location:
frontend/wrangler.toml

Correct file:

```
name = "narmada-billing-production"
compatibility_date = "2026-05-24"

[assets]
directory = "."
```

IMPORTANT:
Frontend worker should NOT contain:

```
main = "src/index.js"
```

because frontend is static files only.

---

8. How Frontend Deployment Works

Steps:

1. Open terminal
2. Go to frontend folder

Command:
```
cd frontend
```

3. Deploy frontend

Command:
```
wrangler deploy
```

After deployment:
Cloudflare uploads:
- index.html
- JS files
- CSS
- images
- assets

Then frontend becomes live.

Frontend URL:
https://narmada-billing-production.jaypatni1710.workers.dev

---

9. How Backend Deployment Works

Steps:

1. Open terminal
2. Go to backend folder

Command:
```
cd backend
```
3. Deploy production backend

```
Command:
wrangler deploy --env production

4. Deploy sandbox backend

Command:
wrangler deploy --env sandbox

Backend handles:
- APIs
- Database
- CRUD operations
```

---

10. Understanding APIs

```
API means communication between frontend and backend.

Frontend calls APIs.
Backend returns data.

Example APIs:

GET /api/stats
Returns dashboard stats.

GET /api/bills
Returns all bills.

GET /api/clients
Returns all clients.

POST /api/clients
Creates new client.

Example:
Dashboard opens → frontend calls /api/stats → backend returns totals.
```

---

11. CORS Explanation

```
CORS is VERY IMPORTANT.

Without CORS:
Frontend cannot call backend.

Error looks like:
Access-Control-Allow-Origin missing

Correct code:

const cors = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type',
};

This allows browser communication.

Without this:
Frontend breaks.
```

---

12. Common Errors and Their Fixes

```
ERROR 1:
localhost:8787 fetch error

Reason:
Frontend still pointing to localhost.

Fix:
Update config.js.

--------------------------------

ERROR 2:
Route not found

Reason:
Wrong API route opened.

Correct examples:
/api/stats
/api/bills
/api/clients

--------------------------------

ERROR 3:
There is nothing here yet

Reason:
Frontend assets missing.

Fix:
Redeploy frontend.

--------------------------------

ERROR 4:
CORS blocked

Reason:
Missing CORS headers.

Fix:
Add Access-Control-Allow-Origin.

--------------------------------

ERROR 5:
Wrong worker deployed

Reason:
Using wrong wrangler.toml.

Fix:
Use correct frontend/backend wrangler files.
```

---

13. Important Workers You Must Keep

```
DO NOT DELETE:

Production:
- narmada-billing-production
- narmada-billing-api-production

Sandbox:
- narmada-billing-sandbox
- narmada-billing-api-sandbox

These are the main workers.

Unused accidental workers can be deleted safely.
```

---

14. Safe Workflow for Future Changes

```
Correct workflow:

1. Make changes in sandbox
2. Test everything
3. Check console errors
4. Verify APIs
5. Verify database
6. Then move to production

Never directly change production first.
```

---

15. Future Improvements

```
Good future features:

- Login system
- User authentication
- Better invoice PDF
- Excel export
- Search filters
- GST automation
- Email invoices
- Backup system
- Better dashboard analytics
- Mobile responsive UI
```

---

16. Final Important Notes

```
This project now has professional structure.

Good things:
- Frontend separated
- Backend separated
- Sandbox separated
- Production separated
- Database separated

This structure is:
- Safer
- Easier to maintain
- Easier to debug
- Easier to scale

Always:
- Keep backups
- Test in sandbox
- Check config.js before deploy
- Use correct wrangler.toml
```